



Universidade Luterana do Brasil

Curso Análise e Desenvolvimento de Sistemas

**IMPLEMENTAÇÃO DE REQUISITOS EM UMA API PARA
GERENCIAMENTO DE CLÍNICA VETERINÁRIA COM .NET E ENTITY
FRAMEWORK CORE**

ANDERSON GOULARTE PATRICIO

TORRES

2025

**IMPLEMENTAÇÃO DE REQUISITOS EM UMA API PARA
GERENCIAMENTO DE CLÍNICA VETERINÁRIA COM .NET E ENTITY
FRAMEWORK CORE**

Artigo Acadêmico, apresentado como um
dos requisitos para avaliação na disciplina
de: Modelagem de Software.

Professor: Lucas Fogaça.

TORRES, RS

2025

SUMÁRIO

1 INTRODUÇÃO	2
1.1 Justificativa	2
1.2 Objetivos.....	3
1.2.1 Objetivo Geral.....	3
2 DESENVOLVIMENTO.....	3
2.1 Requisitos Funcionais.....	3
2.2 Requisitos Não Funcionais:	8
2 CONSIDERAÇÕES FINAIS	10
3 REFERÊNCIAS BIBLIOGRÁFICAS	11

1 INTRODUÇÃO

O presente artigo tem como objetivo descrever detalhadamente o processo de atendimento aos requisitos levantados durante o desenvolvimento da API para Gerenciamento de Clínicas Veterinárias. Esta aplicação foi concebida para facilitar a gestão de clientes (tutores) e seus respectivos animais de estimação (pets), oferecendo funcionalidades de cadastro, atualização, exclusão e consulta de dados. O cenário de desenvolvimento envolveu a utilização da plataforma .NET, com C# como linguagem de programação, e o Entity Framework Core para a camada de persistência de dados com SQLite. Serão explicitados os requisitos funcionais (RF) e não funcionais (RNF) do projeto, demonstrando como cada um foi implementado e validado no código-fonte da aplicação, seguindo boas práticas de desenvolvimento como o padrão Repository e o uso de injeção de dependências.

1.1 Justificativa

A gestão eficiente de informações em clínicas veterinárias é fundamental para a qualidade do atendimento e a satisfação dos clientes. Atualmente, muitas clínicas ainda podem depender de processos manuais ou sistemas legados pouco eficientes para gerenciar dados de tutores, históricos de animais de estimação, agendamentos e outros aspectos operacionais. Essa abordagem pode levar a dificuldades no acesso rápido a informações cruciais, erros no registro de dados, perda de tempo em tarefas administrativas e, conseqüentemente, impactar a capacidade da clínica de oferecer um serviço ágil e personalizado. A crescente demanda por serviços veterinários e a maior

conscientização sobre o bem-estar animal reforçam a necessidade de ferramentas tecnológicas que otimizem esses processos.

Neste contexto, o desenvolvimento de uma API (Interface de Programação de Aplicativos) moderna e bem estruturada para o gerenciamento de clínicas veterinárias se justifica pela necessidade de prover uma solução centralizada, flexível e escalável. Tal sistema permite a organização sistemática dos cadastros de tutores e pets, facilita a consulta e atualização de históricos, e pode servir como base para futuras integrações com outros módulos ou sistemas (como agendamento online, prontuários eletrônicos detalhados, comunicação com clientes, etc.). A utilização de tecnologias atuais como .NET e Entity Framework Core, aliada a boas práticas de desenvolvimento como o padrão Repository e injeção de dependência, garante a criação de uma aplicação robusta, de fácil manutenção e com potencial para evoluir conforme as necessidades do negócio. Portanto, este projeto visa contribuir para a modernização da gestão em clínicas veterinárias, melhorando a eficiência operacional e a qualidade do serviço prestado.

1.2 Objetivos

1.2.1 Objetivo Geral

Desenvolver uma API RESTful para o gerenciamento de clínicas veterinárias, denominada "clinica-veterinaria-api-dotnet", utilizando a plataforma .NET e o Entity Framework Core com SQLite. A aplicação deverá permitir o cadastro, atualização, exclusão e consulta de dados de tutores e seus respectivos animais de estimação, seguindo boas práticas de desenvolvimento de software, como a arquitetura em camadas, o padrão Repository e a injeção de dependências, a fim de fornecer uma solução organizada, eficiente e de fácil manutenção para a gestão das informações essenciais da clínica.

2 DESENVOLVIMENTO

Nesta seção, cada requisito funcional e não funcional é apresentado, seguido de uma análise de sua implementação no sistema "clinica-veterinaria-api-dotnet".

2.1 Requisitos Funcionais

2.1.1 RF01: O sistema deve permitir cadastrar, atualizar, excluir e consultar Tutores.

Descrição: Este requisito abrange o ciclo completo de gerenciamento de dados dos tutores dos animais.

Modelo de Dados (Models/Tutor.cs): A entidade Tutor foi definida para representar os dados do tutor, incluindo propriedades como Id, Nome, Cpf, Email e Telefone.

```
1 public class Tutor
2 {
3     public int Id { get; set; }
4     public string Nome { get; set; } = "";
5     public string Telefone { get; set; } = "";
6     public string Email { get; set; } = "";
7 }
```

Cadastro (POST /api/tutor):

```
1 [HttpPost]
2 public async Task<ActionResult<Tutor>> Post(Tutor tutor)
3 {
4     var novoTutor = await _ITutorRepo.AddAsync(tutor);
5     return CreatedAtAction(nameof(GetById), new { id = novoTutor.Id }, novoTutor);
6 }
```

Atualização (PUT /api/tutor/{id}):

```
1 [HttpPut("{id}")]
2 public async Task<ActionResult> Put(int id, Tutor tutor)
3 {
4     var update = await _ITutorRepo.PutAsync(id, tutor);
5     return update is null ? NotFound() : Ok(update);
6 }
```

Exclusão (DELETE /api/tutor/{id}):

```
1 [HttpDelete("{id}")]
2     public async Task<ActionResult> Delete(int id)
3     {
4         var pet = await _ITutorRepo.DeleteAsync(id);
5         return pet ? NoContent() : NotFound();
6     }
```

Consulta: * Por ID (GET /api/tutor/{id}):

```
1 [HttpGet("{id}")]
2     public async Task<ActionResult<Tutor>> GetById(int id)
3     {
4         var tutor = await _ITutorRepo.GetByIdAsync(id);
5         return tutor is null ? NotFound() : Ok(tutor);
6     }
```

2.1.2 RF02: O sistema deve permitir cadastrar, atualizar, excluir e consultar Pets.

Descrição: Similar ao RF01, este requisito define o gerenciamento completo dos dados dos animais de estimação.

Modelo de Dados (Models/Pet.cs): A entidade Pet foi definida com propriedades como Id, Nome, Especie, Raca, TutorId.

```
1 public class Pet
2 {
3     public int Id { get; set; }
4     public string Nome { get; set; } = "";
5     public string Especie { get; set; } = "";
6     public string Raca { get; set; } = "";
7     public int TutorId { get; set; }
8 }
```

Cadastro (POST /api/pet):

```
1 [HttpPost]
2 public async Task<ActionResult<Pet>> Post(Pet pet)
3 {
4     var novoPet = await _petIRepo.AddAsync(pet);
5     return CreatedAtAction(nameof(GetById), new { id = novoPet.Id }, novoPet);
6 }
```

Atualização (PUT /api/pet/{id}):

```
1 [HttpPut("{id}")]
2 public async Task<ActionResult> Put(int id, Pet pet)
3 {
4     var update = await _petIRepo.PutAsync(id, pet);
5     return update is null ? NotFound() : Ok(update);
6 }
```

Exclusão (DELETE /api/pet/{id}):

```
1 [HttpDelete("{id}")]
2 public async Task<ActionResult> Delete(int id)
3 {
4     var pet = await _petIRepo.DeleteAsync(id);
5     return pet ? NoContent() : NotFound();
6 }
```

Consulta: * Por ID (GET /api/pet/{id}):

```
1 [HttpGet("{id}")]
2     public async Task<ActionResult<Pet>> GetByid(int id)
3     {
4         var pet = await _petIRepo.GetByIdAsync(id);
5         return pet is null ? NotFound() : Ok(pet);
6     }
```

2.1.3 RF03: Cada Pet cadastrado deve pertencer a um Tutor específico.

Descrição: Garante a integridade referencial entre Pets e Tutores.

Algoritmo: A classe Pet (Models/Pet.cs) possui a propriedade TutorId como chave estrangeira para identificar a qual Tutor pertence.

```
1 public class Pet
2 {
3     public int Id { get; set; }
4     public string Nome { get; set; } = "";
5     public string Espécie { get; set; } = "";
6     public string Raca { get; set; } = "";
7     public int TutorId { get; set; }
8 }
```

2.1.4 RF04: O sistema deve validar obrigatoriamente campos importantes como nome e telefone.

Descrição: Garante a integridade referencial entre Pets e Tutores.

Algoritmo:



```
1 using System.ComponentModel.DataAnnotations;
2
3 public class Pet
4 {
5     public int Id { get; set; }
6     public string Nome { get; set; } = "";
7     [Required]
8     public string Especie { get; set; } = "";
9     [Required]
10    public string Raca { get; set; } = "";
11    public int TutorId { get; set; }
12 }
```

2.2 Requisitos Não Funcionais:

2.2.1 RNF01: A aplicação deve usar persistência de dados com Entity Framework Core e SQLite.

Descrição: Define a tecnologia de acesso a dados e o sistema de gerenciamento de banco de dados.

Algoritmo:



```
1 var builder = WebApplication.CreateBuilder(args);
2
3 // Adicionar DbContext
4 builder.Services.AddDbContext<AppDbContext>(options =>
5     options.UseSqlite("Data Source=biblioteca.db"));
```

Código implementado dentro do program.cs



```

1  using Microsoft.EntityFrameworkCore;
2
3  public class AppDbContext : DbContext
4  {
5      public AppDbContext(DbContextOptions<AppDbContext> options)
6          : base(options) { }
7
8      public DbSet<Tutor> Tutor => Set<Tutor>();
9      public DbSet<Pet> Pet => Set<Pet>();
10 }

```

2.2.2 RNF02: Os códigos devem ser disponibilizados via GitHub em um repositório público organizado.

Descrição: Requisito relacionado à gestão de código e colaboração.

O código-fonte da aplicação está versionado e disponível em um repositório público no GitHub, com link <https://github.com/1patricio/clinica-veterinaria-api-dotnet>.

A organização do código segue uma estrutura de pastas lógica, separando responsabilidades: Controllers, Models, Repositories (com uma subpasta Interfaces), Services (usando Interfaces próprias para os serviços), Data (para o DbContext) e Migrations. Esta estrutura promove a clareza e manutenibilidade do projeto.

2.2.3 RNF03: O código deve seguir boas práticas de desenvolvimento, usando o padrão Repository e injeção de dependências.

3 **Descrição:** Requisito relacionado à organização do código e padrão de projeto



```

1  using BibliotecaApi.Repositories.Interfaces;
2  using Microsoft.EntityFrameworkCore;
3  public class TutorRepository : ITutorRepository
4  {
5      private readonly AppDbContext _context;
6      public TutorRepository(AppDbContext context)
7      {
8          _context = context;
9      }

```



```
1 namespace BibliotecaApi.Repositories.Interfaces;
2
3 public interface ITutorRepository
4 {
5     Task<IEnumerable<Tutor>> GetAllAsync();
6     Task<Tutor?> GetByIdAsync(int id);
7     Task<Tutor> AddAsync(Tutor tutor);
8     Task UpdateAsync(Tutor tutor);
9     Task<bool> DeleteAsync(int id);
10    Task<Tutor?> PutAsync(int id, Tutor tutor);
11 }
```



```
1 using BibliotecaApi.Repositories.Interfaces;
2 using Microsoft.EntityFrameworkCore;
3
4 var builder = WebApplication.CreateBuilder(args);
5
6 builder.Services.AddDbContext<AppDbContext>(options =>
7     options.UseSqlite("Data Source=biblioteca.db"));
8
9 builder.Services.AddScoped<IPetRepository, PetRepository>();
10 builder.Services.AddScoped<ITutorRepository, TutorRepository>();
```

2 CONSIDERAÇÕES FINAIS

A implementação da API para Gerenciamento de Clínicas Veterinárias demonstrou a eficácia da plataforma .NET e do Entity Framework Core no desenvolvimento de aplicações robustas e orientadas a dados. O atendimento aos requisitos funcionais foi alcançado através da criação de endpoints RESTful claros e da implementação da lógica de negócios nas camadas de serviço e repositório. Os requisitos não funcionais, como a persistência com SQLite e a adoção de boas práticas de desenvolvimento, como o padrão Repository e a injeção de dependências, foram cruciais para garantir a qualidade, manutenibilidade e testabilidade do código.

Os benefícios do uso das técnicas ensinadas são significativos. A arquitetura em camadas (Controllers, Services, Repositories) promoveu uma clara separação de responsabilidades. O padrão Repository abstraiu a complexidade do acesso a dados, tornando os serviços mais limpos e focados na lógica de negócios. A injeção de dependências simplificou o gerenciamento das dependências entre os componentes e facilitou a escrita de testes unitários. O Entity Framework Core agilizou o desenvolvimento da camada de persistência, permitindo que o foco permanecesse nas regras de negócio da aplicação.

3 REFERÊNCIAS BIBLIOGRÁFICAS

MICROSOFT. Entity Framework Core. Microsoft Docs. Disponível em: <https://docs.microsoft.com/ef/core/>. Acesso em: 27 mai. 2025.